Setting up Python

Topics in Macroeconomics – uv, VS Code and the Week 1 lab

Lukas Boehnert

PhD elective, 2026/27

Bank of Portugal & Católica Lisbon

What we set up, and why

- **Three pieces:**
  1) uv: installs Python and the packages each project needs
  2) VS Code: the editor; runs the lab scripts cell by cell
  3) The lab project: `Week1_Lab/` with a `pyproject.toml` that lists the packages
- **Why uv** (and not Anaconda or `pip install` into the system Python)?
  - One tool for everything, same commands on Mac and Windows
  - Each project gets its own environment (`.venv/`): no version clashes between projects
  - `uv.lock` pins exact versions ⇒ everyone runs the same code on the same packages
  - Fast: the full lab environment installs in a few seconds

⇒ Goal: `week1_simple.py` and `exercise_PT.py` run on your laptop before the lab

Step 1: install uv

Open a terminal. Mac: *Terminal* (Cmd+Space, type "Terminal"). Windows: *PowerShell* (Start menu, type "PowerShell").

Mac (or, with Homebrew: `brew install uv`)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (or: `winget install -id=astral-sh.uv -e`)

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Then close the terminal, open a new one, and check:

```
uv --version
```

- Prints `uv 0.x.y` ⇒ done. "command not found" ⇒ you did not open a new terminal
- No admin rights needed: uv installs into your user folder

Step 2: install Python (with uv)

```
uv python install 3.13
```

- Downloads a standalone Python 3.13 that uv manages
- Does not touch any Python you already have (Anaconda, python.org, the Mac system Python)
- Strictly optional: `uv sync` in Step 4 installs the right Python by itself if it is missing

Check:

```
uv python list --only-installed
```

$\Rightarrow$ You never need to type `python` or `pip` directly: `uv run` picks the right one

Step 3: install VS Code and two extensions

- **Download** from code.visualstudio.com
  - Mac: unzip, drag *Visual Studio Code* into *Applications*
  - Windows: run the installer; tick *"Add to PATH"* and *"Open with Code"*
- **Extensions** (left bar, the four squares, or `Cmd/Ctrl+Shift+X`): install
  - Python (Microsoft)
  - Jupyter (Microsoft): runs the `# %%` cells
- **Built-in terminal:** *View* → *Terminal* (`Ctrl+‘`). It opens in the folder you have open, so all commands below can be typed there
- Mac only, optional: `Cmd+Shift+P` → "Shell Command: Install 'code' command in PATH", so that `code .` opens a folder from the terminal

Step 4: set up the lab project

Unzip `Week1_Lab` somewhere outside OneDrive/iCloud/Dropbox (e.g. `Documents/phd/`), then in VS Code: *File* → *Open Folder...* → `Week1_Lab`.

In the VS Code terminal:

```
uv sync
```

- Reads `pyproject.toml` and `uv.lock`
- Creates `.venv/` with Python 3.13 and the exact package versions
- Run it again whenever `pyproject.toml` changes

What is in the folder

```
Week1_Lab/
  pyproject.toml   <- packages
  uv.lock          <- exact versions
  .python-version  <- 3.13
  week1_simple.py
  week1_fast.py
  exercise_PT.py
  data/
    CTV_DATA.xlsx
    pt_monthly.csv
```

`pyproject.toml` (the part that matters)

```
dependencies = ["numpy", "pandas", "openpyxl",
  "matplotlib", "statsmodels", "linearmodels"]
[dependency-groups]
dev = ["ipykernel"]  # VS Code cells
```

Step 5: run the scripts from the terminal

```
uv run week1_simple.py
uv run exercise_PT.py
```

- `uv run` = "run with this project's `.venv`" (and syncs it first if needed)
- Each takes about a minute (the bootstrap and Monte Carlo cells are the slow part)
- Results print in the terminal; the figures are saved to
  - `figures_simple/` (`week1_simple.py`: the lecture slides, L1–C5)
  - `figures_exercise/` (`exercise_PT.py`: Portugal, MP and CBI shocks)
- **Check it worked:** the last lines of `week1_simple.py` print

```
C5 MA(1) with theta = 2, h = 0..3: truth [1, 2, 0, 0]
  AR(12)      : [1.   0.5  0.   0.01]
  LP on eta_t : [1.   2.   0.   0.01]
```

Step 6: run cell by cell in VS Code (how we use it in class)

- The scripts are split into cells by `# %%` lines: one cell per slide
- **Choose the environment once:** `Cmd/Ctrl+Shift+P` → "Python: Select Interpreter" → the one in `./.venv` (marked *Recommended*)
- **Run a cell:** click inside it and press `Shift+Enter`, or click *Run Cell* above the `# %%` line
  - The *Interactive* window opens on the right; figures appear under each cell
  - First time: if asked for a kernel, pick `.venv (Python 3.13)`
  - Variables stay in memory: change a line, re-run just that cell
- **Always run the *Setup* cell first** (imports, helpers), then the *Data* cell, then any step
- Restart (button at the top of the Interactive window) if things get confused

Your own project (for the class project)

Start a new project

```
uv init my_project
cd my_project
uv add numpy pandas matplotlib statsmodels
uv add --dev ipykernel
uv run main.py
```

- `uv add` writes the package into `pyproject.toml` and `uv.lock`
- `uv remove pkg` takes it out again

One-off scripts, no project

```
uv run --with numpy,pandas script.py
```

- Builds a throw-away environment just for this run

Share your code

- Send `pyproject.toml` + `uv.lock` with your scripts, not `.venv/`
- The other person runs `uv sync` and gets the same versions

If something goes wrong

- `uv: command not found` ⇒ open a new terminal (and restart VS Code) after installing uv
- `ModuleNotFoundError: No module named 'linearmodels'` ⇒ you ran the wrong Python. Use `uv run ...`, or select the `.venv` interpreter (Step 6)
- `FileNotFoundError: data/...` ⇒ open the `Week1_Lab` folder itself in VS Code, not its parent
- `cd` **fails on a path with spaces** ⇒ put it in quotes: `cd "C:\Users\ana\My PhD\Week1_Lab"`
- **Windows: "running scripts is disabled"** ⇒ use the exact install line from Step 1 (it sets `-ExecutionPolicy ByPass`)
- **VS Code does not offer** `.venv` ⇒ run `uv sync` first, then `Cmd/Ctrl+Shift+P` → "Developer: Reload Window"
- **Very slow, or OneDrive/iCloud warnings** ⇒ `.venv/` is thousands of files; move the folder out of the synced folder
- **Figures do not show** when you run the whole file with `uv run`: by design, they are only saved. Use the cells (Step 6) to see them

Cheat sheet

	Mac (Terminal)	**Windows** (PowerShell)
Install uv	see Step 1 (one line, different on Mac and Windows)	
Install Python	`uv python install 3.13`	
Set up the lab	`cd Week1_Lab` then `uv sync`	
Run a script	`uv run week1_simple.py`	`uv run exercise_PT.py`
Add a package	`uv add seaborn`	
Update uv	`uv self update`	
Command palette	`Cmd+Shift+P`	`Ctrl+Shift+P`
Run cell	`Shift+Enter`	`Shift+Enter`
Terminal in VS Code	`Ctrl+‘`	`Ctrl+‘`
Environment's Python	`.venv/bin/python`	`.venv\Scripts\python.exe`

⇒ Please do Steps 1–5 before the first session, and email me if one fails (with a screenshot of the error)